clicker.cs.illinois.edu

Q1

~Code~
340

CS 340

Authentication

Updates

1. Prof. Schatz is still sick :(no office hours today
2. MP 10 due 4/28
3. Exam 3 on 4/30
   a. Study guide released on website.
   b. Practice exam released by Friday.
4. CA applications due April 27th
5. MP 8 is graded, please file an issue on prairie learn if you think there was a mistake.
6. We have class next Tuesday and Thursday (even with the exam).
   a. Tuesday - Final project introduction and review
   b. Thursday - Luther Tychonievich guest speaker (with extra credit). Ask him anything about systems!
7. See course calendar for deadlines and schedule

Learning Goals

Differentiate (4) common cyber attacks, when they are used, and the strategy to counter them.

When/how to use hashing and salting techniques.

How symmetric / asymmetric encryption secures your connections.

Be able to apply the Diffie-Hellman algorithm to get a secure key

NOT: To implement your own cryptography code!!!

Agenda

Accounts & Logins

- Brute force, rainbow table attacks
- Hashing, salting passwords

Randomness

- True vs pseudo random numbers

Hiding Information

- Man-in-the-middle attacks
- Symmetric vs Asymmetric encryption
- Diffie-Hellman key exchange

Storing Passwords Idea I

Login → Pass2002

HTTP Request

```
method: "PUT"

headers: "application/json"

body: {
        username: "Anton",
        password: "pass2002"
}
```

Internet

Database ??

User	Password
Anton	pass2002

Storing Passwords Idea I

Cs340BankAccounts.com

HTTP Request

```
method: "PUT"

headers: "application/json"

body: {
        username: "Anton",
        password: "pass2002"
}
```

Internet

Database

User	Password
Anton	pass2002
Bob11	cs340
Adrian	myparsword
Dell	keyboard

Don't store plaintext!

Hash Function

1. Fixed length output

$h(m) = H$

0-N bytes M bytes

$h(m) \longrightarrow H$

Ex. Hashing with MD5 in python

2. One-way, very hard to reverse!

hashlib.md5(b'**pass1234**') ⟹ 'b4af8040095cb036a4ccdc33431ef9ac9'

Storing Passwords Idea II

More Secure!

Lets hash your passwords!

hashlib.md5(b'**pass2002**')

HTTP Request

```
method: "PUT"

headers: "application/json"

body: {
        username: "Anton",
        password: "pass2002"
}
```

Internet

Database

User	Hash
Anton	412b2d99db5f3ee8c5e2c7a7001a6dfa

|| ??

Log.n → n(pass2002)

clicker.cs.illinois.edu

Q2

~Code~
340

Which of the following is a vulnerability of just using hashing for password storing?

A. If someone gets access to the database of passwords your password is automatically compromised.

A. If another user has the same password as you and someone figures out that password, your password is compromised.

A. If the hashing function is very slow.

Storing Passwords Idea II Vulnerability

Brute force attacks

$h(m) = H$

Try all m until you get H

Slow!

Speed up:

- Only search common m
- Multithreading

Rainbow tables

PB, or TB

Fast!

Size tradeoff

common

Pass	Hash
temp	b57x
⋮	⋮

Salt

Goal: Don’t let me use my rainbow tables!

Salting - Append a random string to your password

pass1234 $\xrightarrow{\text{MD5}}$ b4af804009cb036a4ccdc33431ef9ac9

pass1234 + **av9$0x3n0xc!#11** $\xrightarrow{\text{MD5}}$ b2db5a5469a6acb325efa67fdec8fc89

Salt

Assume a database hasn't salted their passwords. What attack should you use?

clicker.cs.illinois.edu

Q3

~Code~
340

A. Rainbow table. A successful lookup is faster than brute force search.

A. Rainbow table. The different colors will make cracking the passwords more fun and engaging!

A. Brute force. We can divide the search over multiple threads.

A. Brute force. Storing a rainbow table can take up a lot of memory.

clicker.cs.illinois.edu

Q4

~Code~
340

Assume a database has salted their passwords. You have 2000 threads, but only 5GB of memory. What attack should you use?

A. Rainbow table. A successful lookup is faster than brute force search.

A. Brute force. We can divide the search over multiple threads.

A. Brute force. Storing a rainbow table can take up a lot of memory.

A. De-salination attack

Storing Passwords Idea III

Lets salt + hash your passwords!

salt = secrets.token_hex(8)

HTTP Request

```
method: "PUT"

headers: "application/json"

body: {
        username: "Anton",
        password: "pass2002"
}
```

Internet

Database

User	Hash	Salt
Anton		d9!C8v92

Storing Passwords Idea III

Lets salt + hash your passwords!

Login → h(pass2002 + Salt)

!! ??

HTTP Request

```
method: "PUT"

headers: "application/json"

body: {
        username: "Anton",
        password: "pass2002"
}
```

Internet

Database

User	Hash	Salt
Anton	1aa8d8f6e50e8206e bfe2f7609d561b1	d9!C8v92

hashlib.md5(b'**pass2002+d9!C8v92**')

Randomness

Random numbers are essential for security!

pass, salt

How do computers make random numbers?

Pseudorandom

! Pattern !

X

True random

Unpredictable

Security

Which sequence is least likely to be generated by a truly random number generator?

clicker.cs.illinois.edu

Q5

~Code~
340

A. 1, 2, 4, 8

A. 1, 1, 1, 1

A. 9, 8, 3, 7

A. All the same likelihood

Imagine you are a computer. How do you generate secure random numbers for cryptography?

clicker.cs.illinois.edu

Q6

~Code~
340

A. Ask your user to give you a random input

A. Make an API call to ChatGPT

A. Count how many times a user has pressed the ';' key in the last 24 hours.

A. Get robot arms that you can roll dice with, record the results, and analyze the result using the OpenCV library.

Randomness

True random number generators, use **entropy harvesting**!!!

Harvest OS events (hardware interrupts, network traffic)

For a secure database salts need to be random - do they need to be unique?

clicker.cs.illinois.edu

Q7

~Code~
340

A. No. Randomness is enough

A. Yes. If two users have the same final hash and same salt you could crack the full hash function.

A. Yes. If two users have the same final hash and same salt both of their passwords are compromised.

Is The Server Enough?

! Man-in-the-middle !

Idea: Hash?

Un-hash?

HTTP Request

method: "PUT"

headers: "application/json"

body: {
username: "Anton",
password: "pass2002"
}

Internet

HTTP Request

method: "PUT"

headers: "application/json"

body: {
username: "Anton",
password: "pass2002"
}

HTTP Request

method: "PUT"

headers: "application/json"

body: {
username: "Anton",
password: "pass2002"
}

!!!

Database

User	Anton
Hash	1aa8d8f6e50e8206ebf e2f7609d561b1
Salt	d9!C8v92

Idea: Encryption

Hiding Information

Encrypted

"uijt dmbtt jt uif cftu"

k

Decrypted

" This class is the best"

k

key

k = -1

A	B	C	D	E	F	G	H	I	J
1	2	3	4	5	6	7	8	9	10

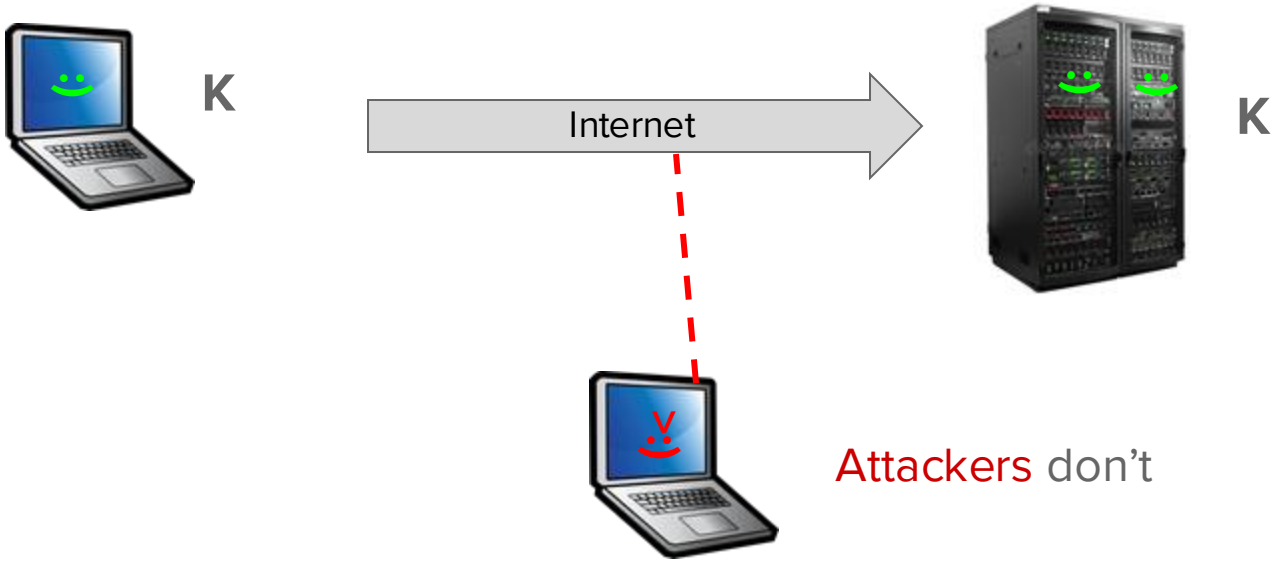
K	L	M	N	O	P	Q	R	S	T
11	12	13	14	15	16	17	18	19	20

U	V	W	X	Y	Z
21	22	23	24	25	26

Symmetric Encryption

Key K can be used to encrypt or decrypt a message

Trusted computers have key K



Note: Hashing vs Encryption

One-way

Hard to reverse

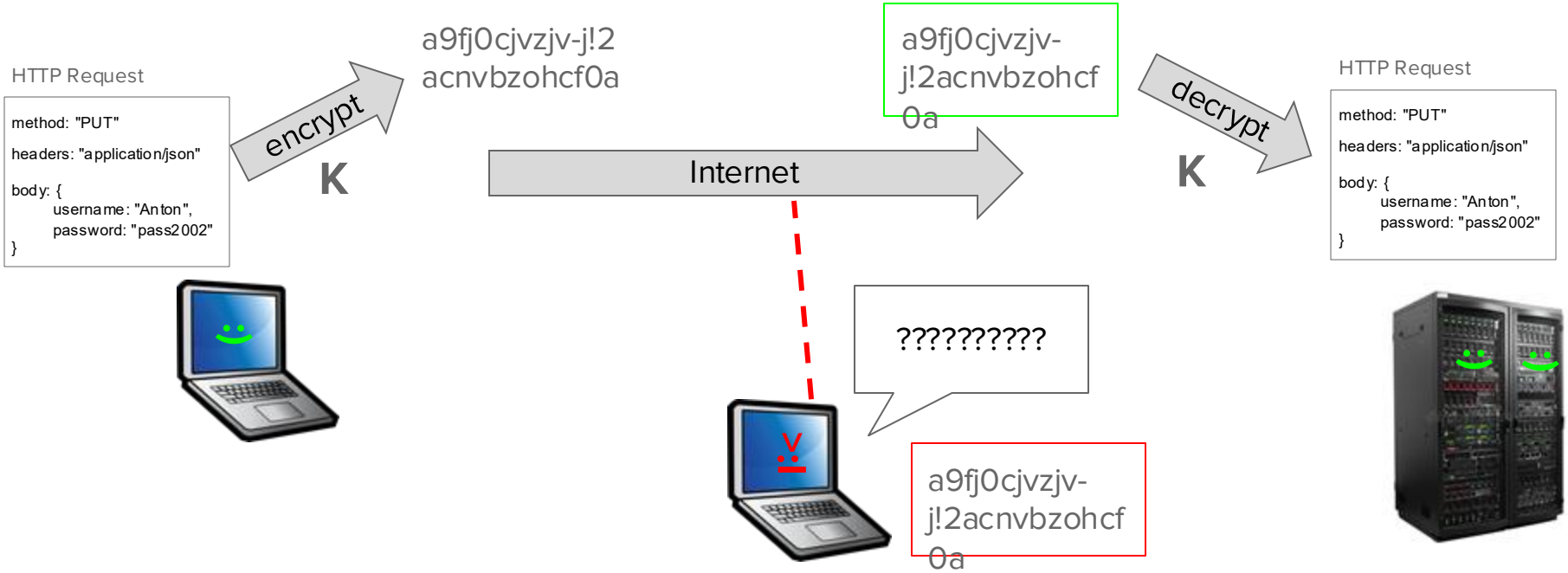
Integrity

Two-way

encrypt / decrypt

Privacy

Symmetric Encryption

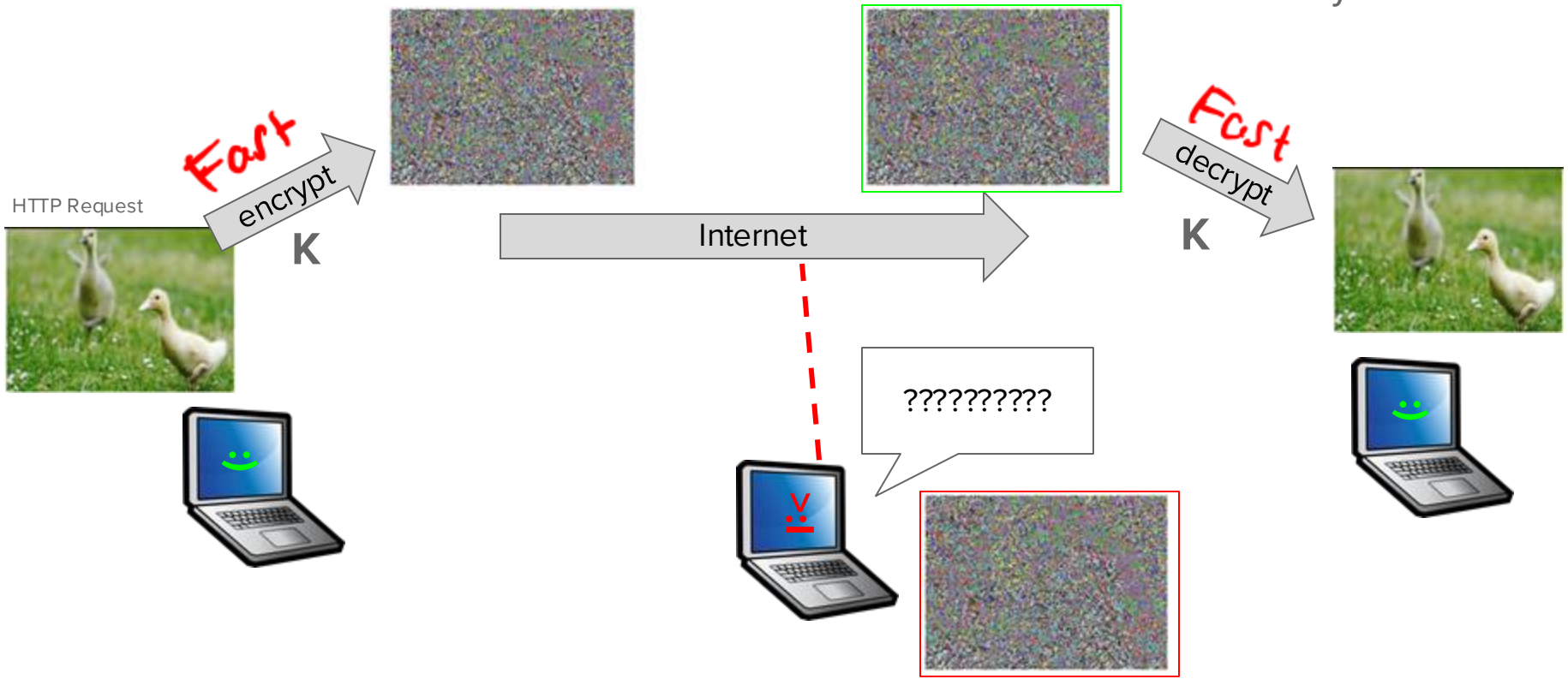


Video Encryption

Live - Streaming

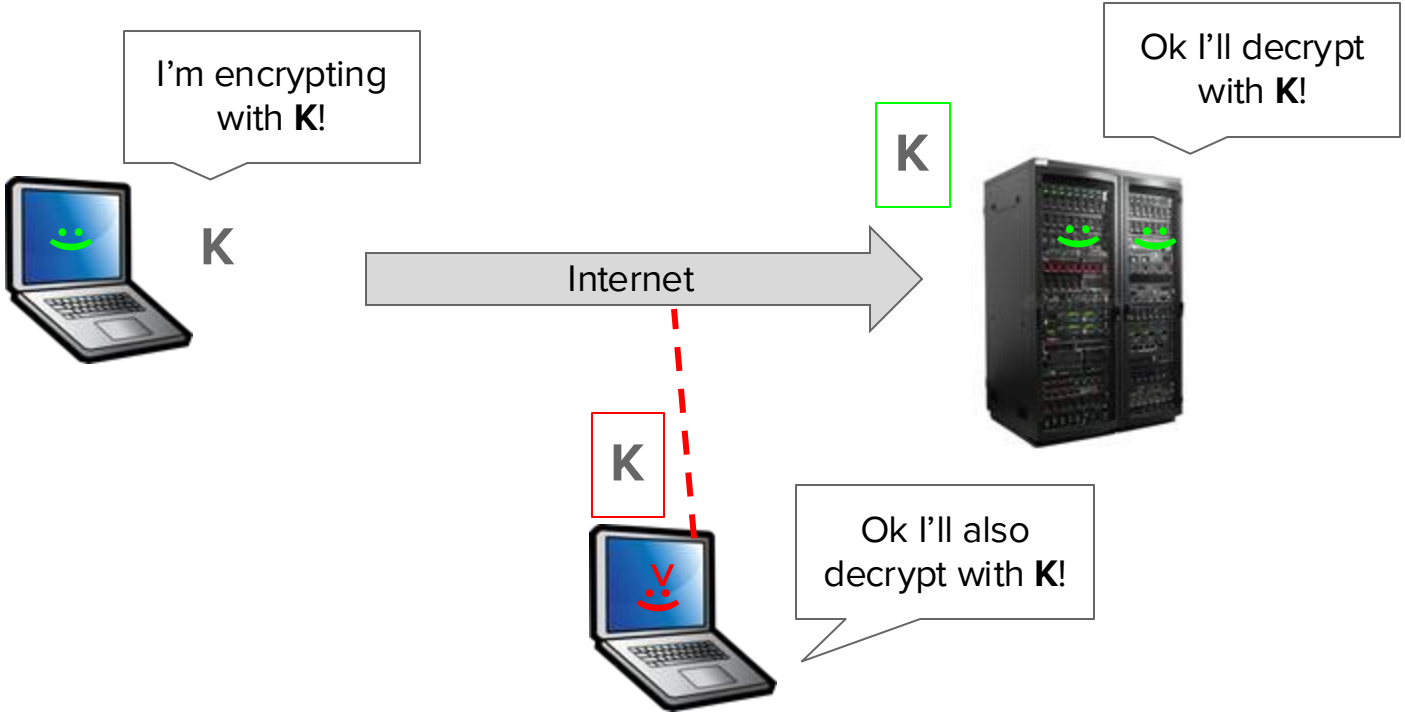
ex. AES

Symmetric



Symmetric Encryption Problem!

How do we agree on a key without an attacker overhearing?



Key Exchange

Agree on k w/out sharing over network

ex. Diffie-Hellman

Asymmetric

Private

Alice

$a = 4$

Public

$p = 23, g = 5$

$A = 4, B = 10$

Bob

$b = 3$

① $A = g^a \bmod p = 4$

$B = g^b \bmod p = 10$

② $k = B^a \bmod p = 18$

$k = A^b \bmod p = 18$

Agree!

!Free exam answer!

clicker.cs.illinois.edu

Q8

~Code~
340

Anton

$A = 26$

Public

$p = 89$, $g = 7$

Prof. Schatz/Otto

$B = 2$

① $A = g^a \bmod p$

② $k = B^a \bmod p$

③ Encrypted Message

42

Message: 2nd MCQ answer, A=1, B=2, C=3, D=4

Symmetric encryption strategy: k + Message

You and your friend are trying to set up a network for securely messaging each other from across the CS340 classroom. What's the best way to avoid a man in the middle attack?

clicker.cs.illinois.edu

Q9

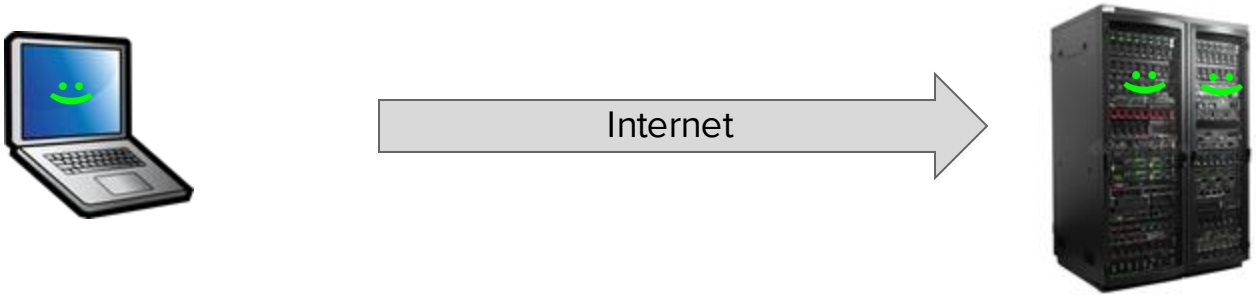
~Code~
340

A. Shouting the key across the room.

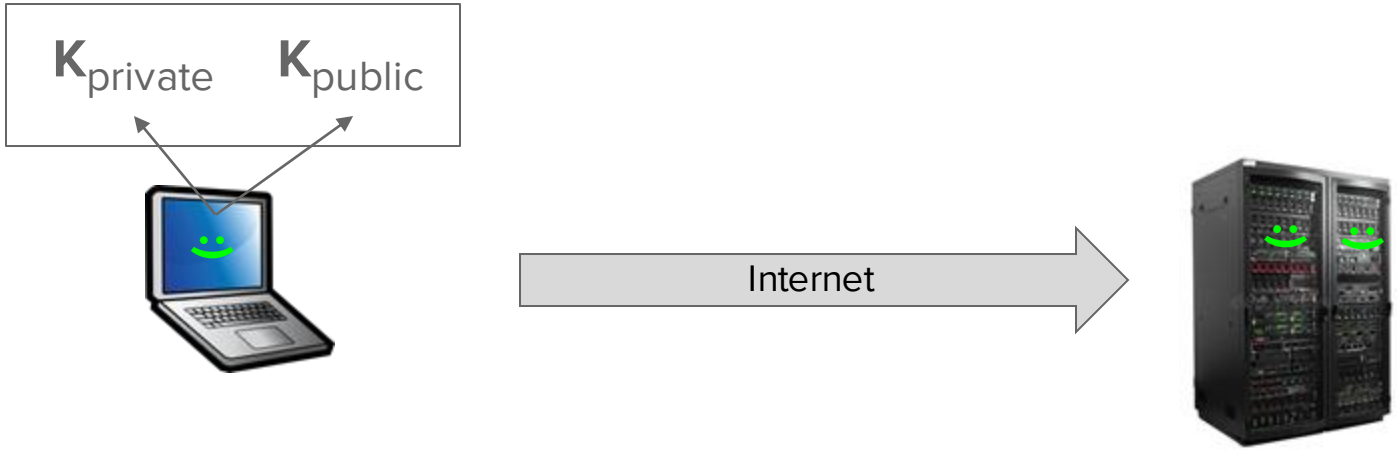
A. Share your key over the network.

A. Use Diffie-Hellman to agree on a key over the network.

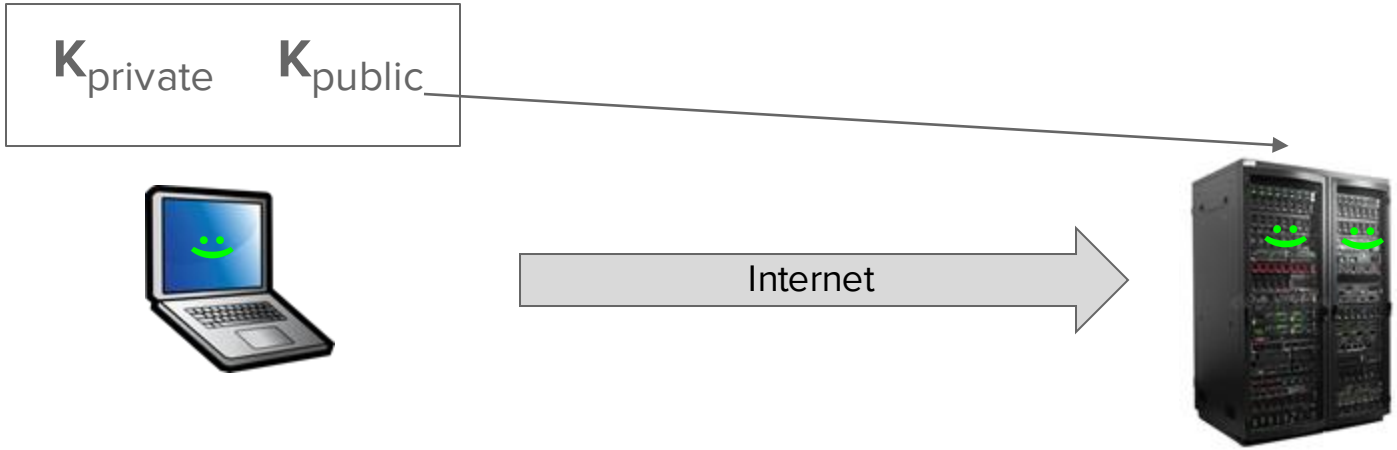
Asymmetric Encryption



Asymmetric Encryption



Asymmetric Encryption



Asymmetric Encryption

$K_{private}$ K_{public}

Internet

K_{public}

Asymmetric Encryption

$K_{private}$ K_{public}

Internet

K_{public}

You use use $K_{private}$ to encrypt

Asymmetric Encryption

$K_{private}$ K_{public}

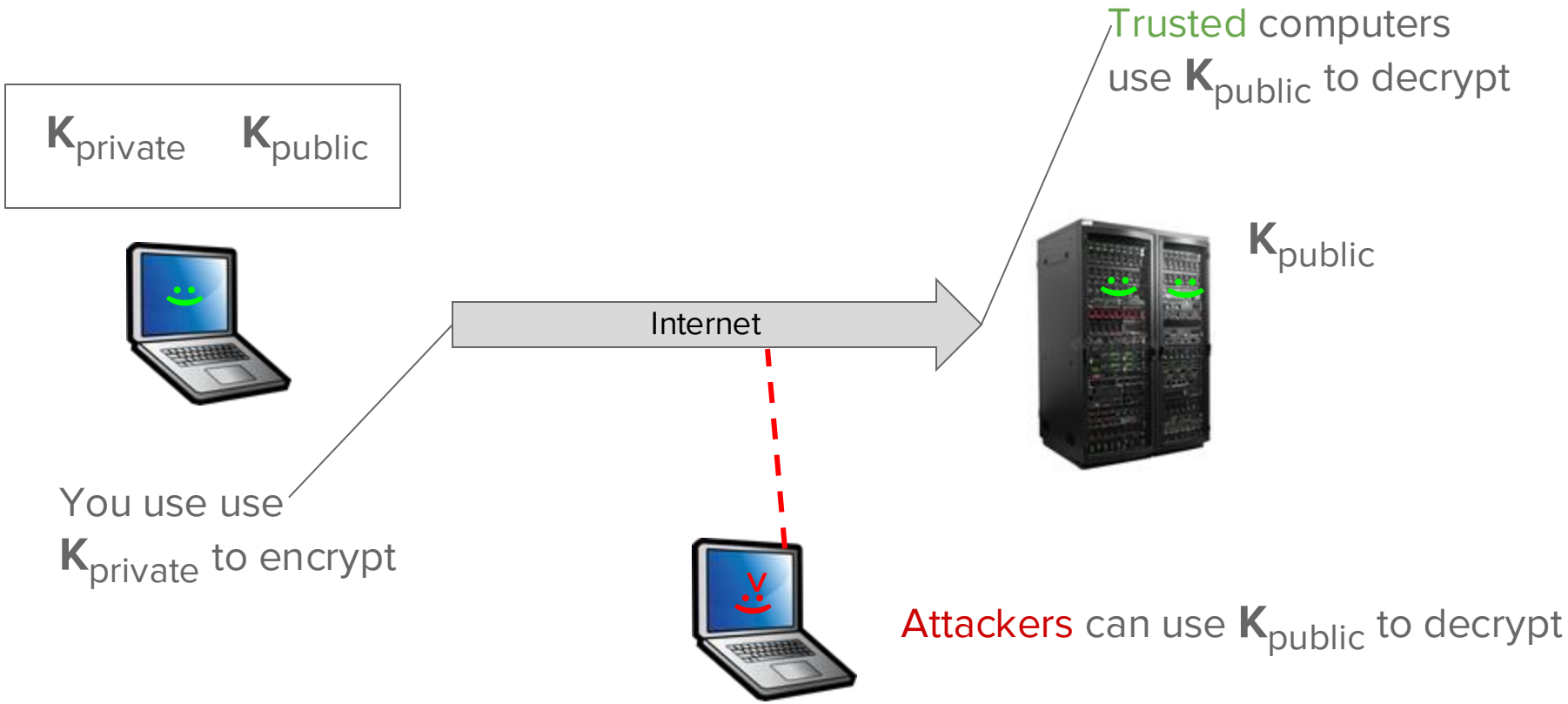
Internet

Trusted computers use K_{public} to decrypt

K_{public}

You use use $K_{private}$ to encrypt

Asymmetric Encryption



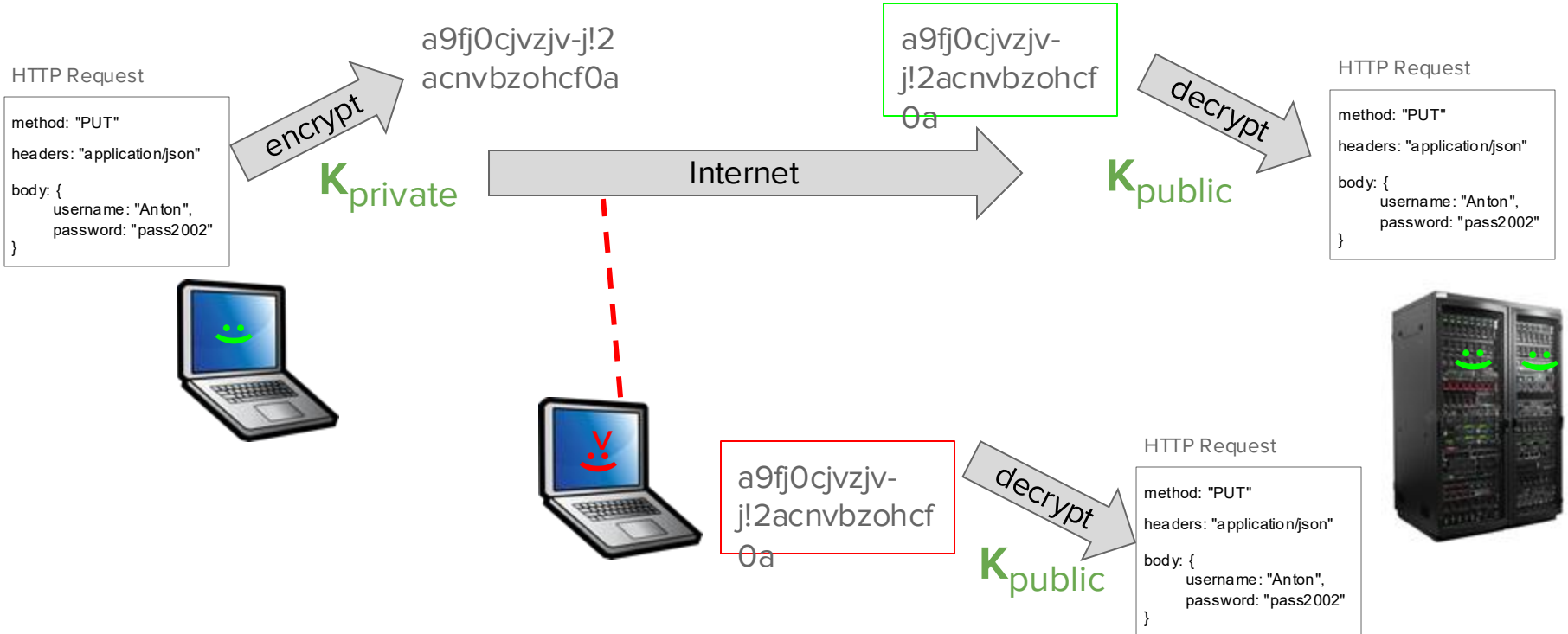
Asymmetric Encryption

Public and private keys only work in pairs!

K^{my}_{public} decrypts $K^{my}_{private}$

K^{my}_{public} <u>wont</u> decrypt $K^{other}_{private}$

Asymmetric Encryption



Digital Signatures

"I am who I say I am"

ex. RSA

Asymmetric

HTTP Request

```
method: "PUT"

headers: "application/json"

body: {
        username: "Anton",
        password: "pass2002"
}
```

encrypt

$K_{private}$

a9fj0cjvzjv-j!2
acnvbzohcf0a

decrypt

K_{public}

HTTP Request

```
method: "PUT"

headers: "application/json"

body: {
        username: "Anton",
        password: "pass2002"
}
```

Digital Signatures

ex. RSA

Asymmetric

Evil HTTP Request

method: "PUT"

headers: "application/json"

body: {
username: "Anton",
password: "HackingYourDatabase"
}

encrypt

$K_{private}$

llasdf932zxcv-
cvkj2 n#fnkbfa

decrypt

K_{public}

HTTP Request

Afj"l: ajf39cl

zcvlj_+ "adjf2922dfxcvz00

31dhaf9h: {
asdfe: "lllckcvi",
cvczv: "[;;ae[ef"
}

???!?!???!?!???!?

Private key acts as a digital ID!

Symmetric vs Asymmetric

Fast
(real-time)

→ Privacy

Slower

→ Identity certificates

→ Diffie-Hellman